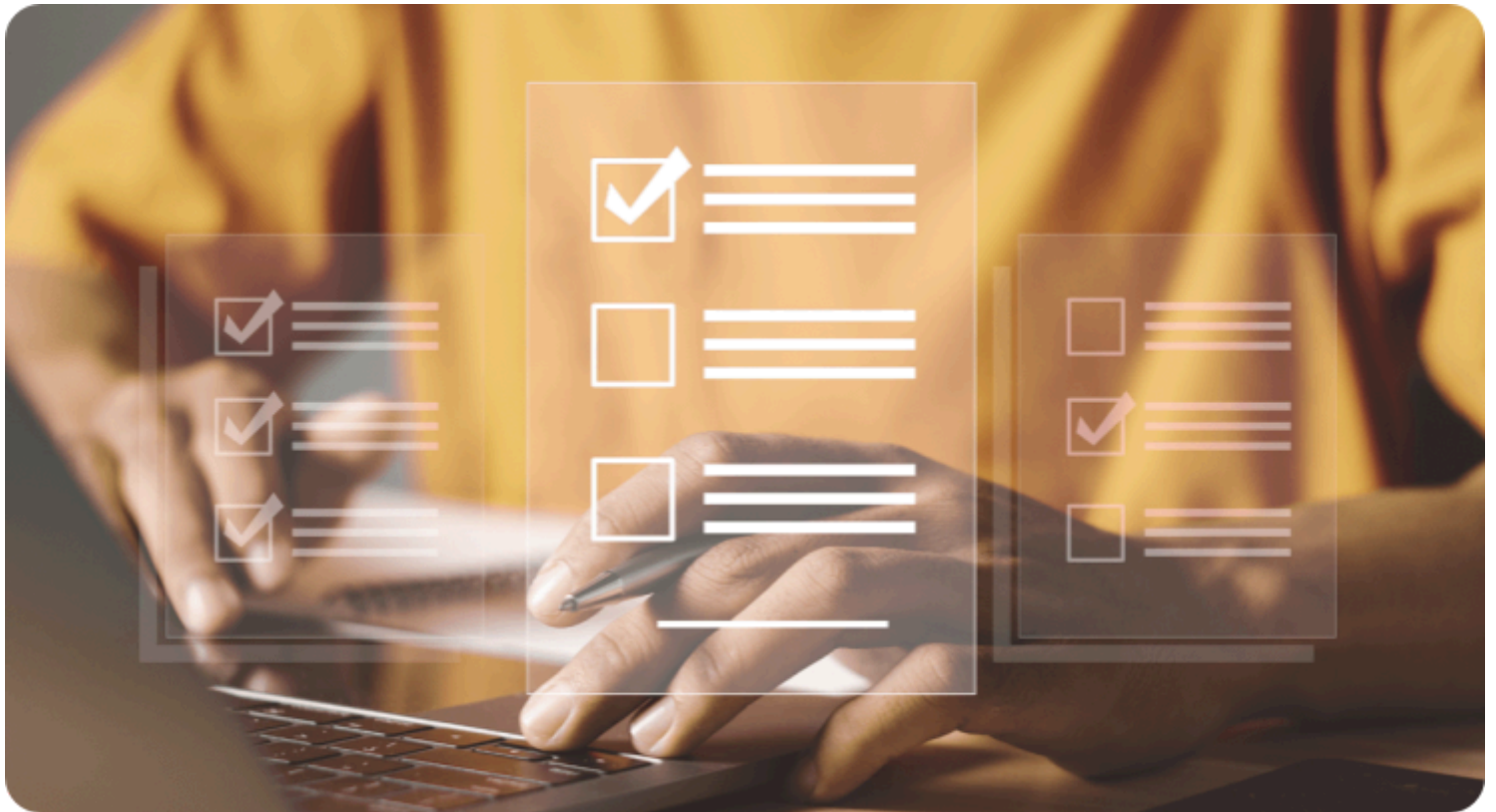




Semana 2

Desarrollo Backend III

(PBY2203)

Instrucciones Específicas

Descripción de la actividad

En esta actividad, implementarás un sistema de migración de procesos batch utilizando Spring Batch, con el objetivo de modernizar el sistema legacy del Banco XYZ. El enfoque será recrear tres procesos clave: el reporte de transacciones diarias, el cálculo de intereses mensuales y la generación de estados de cuenta anuales. El proyecto debe cumplir con los requisitos para garantizar la integridad y consistencia de los datos, aplicando buenas prácticas en la gestión de errores y optimización del rendimiento.

Instrucciones específicas

En función de la actividad formativa de la semana 1 llamada Analizando la arquitectura batch para procesar datos, deberás avanzar en el proyecto en su continuidad, realizando:

1. Configurar un Proyecto Spring Batch

Inicia un proyecto utilizando Spring Batch y configura Jobs y Steps para leer, procesar y escribir los datos de los archivos CSV. Versiónalo en el repositorio de proyectos de GitHub bajo una cuenta de tu propiedad.

2. Implementar un Procesamiento de Datos

Lee los archivos CSV. Aplica transformaciones y validaciones con un ItemProcessor para corregir o manejar los errores en los datos.

Escribe los datos procesados en una base de datos relacional. Puedes elegir entre PostgreSQL, MySQL u Oracle.

3. Manejo de Errores y Excepciones:

Deberás tener en cuenta que la data a ser procesada podría contener datos incorrectos y mal clasificados, por ello deberás manejar reglas para asegurar su consistencia.

4. Implementar políticas personalizadas y tolerancia a fallos:

Deberás tener en cuenta que deberás considerar políticas personalizadas para poder implementar una correcta tolerancia a fallos.

5. Implementar políticas de escalamiento:

Deberás tener en cuenta que deberás considerar políticas de escalamiento para poder utilizar 3 hilos de ejecución paralela con chunks de tamaño 5.

Requerimientos del Sistema

Basándote en los datos de https://github.com/KariVillagran/bank_legacy_data deberás reescribir los procesos en Spring Batch.

Configura un Job para cada uno de los tres procesos:

- **Reporte de Transacciones Diarias:** Procesar transacciones diarias para detectar anomalías y generar un resumen.
- **Cálculo de Intereses Mensuales:** Aplicar intereses sobre cuentas de ahorro y préstamos, y actualizar el saldo final en una base de datos.
- **Generación de Estados de Cuenta Anuales:** Compilar datos anuales para cada cuenta y generar un informe detallado para auditorías.

Entrega

- **Código Fuente:** Sube el código del proyecto a un repositorio de GitHub con su propuesta técnica
- **Documentación (README.md):** Describe el objetivo del proyecto, la estructura del código y las instrucciones para ejecutar el proyecto.
- **Evidencia de Ejecución:** Proporciona capturas de pantalla o salidas de consola mostrando la ejecución de cada Job y los resultados generados.

Asegúrate de adjuntar todos estos documentos y componentes requeridos, todos deben estar en una misma carpeta. Finalmente, adjunta el archivo al subir tu actividad, asegurándote de que esté comprimido con la siguiente nomenclatura:

ExpX_Sx_nombre_del_grupoX. Ejemplo: Exp1_S2_Grupo4

IMPLEMENTACIÓN DE SISTEMA DE MIGRACIÓN BATCH CON SPRING BATCH PARA BANCO XYZ

1. Introducción

Cuando las empresas crecen y manejan un volumen alto de operaciones, los sistemas antiguos o legados suelen quedarse cortos. En el caso del Banco XYZ, procesar registros diarios desde archivos planos (CSV) de forma manual o con scripts básicos ya no era viable por temas de rendimiento y el riesgo de que un fallo tirara todo el proceso abajo.

Para solucionar esto, nos propusimos construir una solución moderna y robusta utilizando **Spring Boot** y **Spring Batch**. La idea principal fue automatizar la migración de toda esa data hacia una base de datos relacional en MySQL. En este desarrollo aplicamos conceptos clave de la ingeniería de software moderna: procesar la información por bloques (*chunks*), levantar hilos en paralelo (*multithreading*) para que vuele el procesamiento, y agregar listeners y políticas de tolerancia a errores para asegurarnos de que si algo falla, sepamos exactamente dónde y por qué sin corromper la base de datos.

2. Objetivos

2.1 Objetivo General

Diseñar, desarrollar y poner a prueba un sistema de migración automatizado con Spring Batch, capaz de procesar de manera concurrente, eficiente y segura la información de cuentas y transacciones del Banco XYZ.

2.2 Objetivos Específicos

- Organizar el código en una estructura modular limpia en Spring Boot, separando claramente las responsabilidades entre la lectura, el procesamiento y la escritura de los datos.
- Configurar el flujo de trabajo en bloques controlados de registros (*chunks*) para optimizar el uso de memoria.
- Implementar concurrencia mediante un pool de hilos personalizado (`ThreadPoolTaskExecutor`), logrando que los *steps* corran en paralelo.

- Integrar un sistema de monitoreo en consola usando *Job Execution Listeners* para medir tiempos y revisar el estado exacto de cada ejecución.

3. Desarrollo del Proyecto

3.1 Arquitectura y Componentes

Para mantener el orden y facilitar las buenas prácticas de desarrollo backend, distribuimos el proyecto en los siguientes paquetes principales:

- **config**: Aquí configuramos los beans de Spring Batch, la conexión a la base de datos y la administración de los hilos concurrentes.
- **controller**: Contiene el JobController, que nos expone rutas REST simples para lanzar cada proceso de migración cuando lo necesitemos.
- **reader, processor, writer**: Son las piezas clave que se encargan de leer desde los CSV, transformar los datos a nuestras entidades de negocio y guardarlos en MySQL.
- **listener**: Aloja el BancoJobListener, nuestra herramienta para llevar el registro en consola del ciclo de vida de los jobs.

3.2 Configuración del Procesamiento Concurrente y Chunks

Siguiendo los requerimientos de optimización, ajustamos el sistema para que trabaje con bloques de **5 registros** (chunk(5)). Además, integramos un ThreadPoolTaskExecutor configurado con **3 hilos**, lo que nos permitió distribuir la carga de trabajo y aprovechar al máximo la ejecución concurrente.

3.3 Jobs Implementados

Creamos tres procesos batch independientes para cubrir las áreas operativas solicitadas:

- **interesJob**: Toma el archivo intereses.csv, calcula y aplica las tasas mensuales que corresponden a cada cuenta.
- **transaccionJob**: Lee y valida las transacciones financieras del día a partir de transacciones.csv.
- **cuentaAnualJob**: Agrupa y consolida los estados de cuenta anuales desde cuentas_anuales.csv para dejar todo listo para auditorías.

4. Resultados y Evidencias de Ejecución

Al probar los servicios a través de las peticiones en el controlador REST, el comportamiento fue excelente. En la consola pudimos ver cómo los hilos (batch-thread-1, batch-thread-2, batch-thread-3) se encargaron de procesar los datos de forma paralela hasta arrojar el estado **COMPLETED**.

Por otro lado, al revisar directamente en MySQL Workbench sobre el esquema bancoxyz_db, las consultas de verificación nos confirmaron que los registros se insertaron de forma limpia y completa en sus respectivas tablas (transacciones, intereses, cuentas_anuales), validando que la migración fue 100% exitosa.

5. Conclusiones

Desarrollar este sistema de migración con Spring Batch nos demostró el gran poder que tiene este framework para resolver problemas complejos de integración de datos. Trabajar con una arquitectura basada en *chunks* y sumarle ejecución multihilo reduce drásticamente los tiempos de procesamiento y cuida los recursos del servidor.

Además, la modularidad que logramos en el código y el uso de los *listeners* hacen que la aplicación sea muy fácil de mantener, escalar y adaptar ante cualquier cambio o error que traigan los archivos de origen en el futuro.

6. Anexos: Evidencias de Validación del Sistema

A continuación, se presentan las capturas de pantalla que validan el correcto funcionamiento y la persistencia de los datos en cada una de las capas del sistema.

6.1 Ejecución Concurrente y Estados COMPLETED en Consola Las siguientes capturas demuestran el funcionamiento del `ThreadPoolTaskExecutor` utilizando múltiples hilos (`batch-thread-1`, `batch-thread-2`, `batch-thread-3`), la monitorización mediante el `BancoJobListener` y el cierre exitoso de los procesos:

Parte 1 (Inicio y Cuentas Anuales):

```
2026-08-20T08:53:46:280-04:00 INFO 32894 --- [main] j.LocalContainerEntityManagerFactoryBean : Initialized JPA EntityManagerFactory for persistence unit 'default'
2026-08-20T08:53:46:366-04:00 WARN 32894 --- [main] o.s.d.c.c.s.b.FaultTolerantStepBuilder : Asynchronous TaskExecutor detected with ItemStream reader. This is probably an error, and may lead to incorrect restart data being stored.
2026-08-20T08:53:46:421-04:00 WARN 32894 --- [main] o.s.d.c.c.s.b.FaultTolerantStepBuilder : Asynchronous TaskExecutor detected with ItemStream reader. This is probably an error, and may lead to incorrect restart data being stored.
2026-08-20T08:53:46:652-04:00 WARN 32894 --- [main] jpahibernate.Configuration.jpahibernateConfiguration : spring.jpa.open-in-view is enabled by default. Therefore, database queries may be performed during view rendering. Explicitly configure spring.jpa.open-in-view to disable this warning
2026-08-20T08:53:47:013-04:00 INFO 32894 --- [main] o.s.b.w.embedded.tomcat.Tomcat8WebServer : Tomcat started on port 8080 (http) with context path '/'
2026-08-20T08:53:47:022-04:00 INFO 32894 --- [main] o.d.migration.MigrationMatchApplication : Started MigrationMatchApplication in 4.469 seconds (process running for 4.83)
2026-08-20T08:55:49:299-04:00 INFO 32894 --- [nio-8080-exec-1] o.a.e.c.c.[Tomcat].[localhost].[/] : Initializing Spring DispatcherServlet 'dispatcherServlet'
2026-08-20T08:55:49:299-04:00 INFO 32894 --- [nio-8080-exec-1] o.a.w.servlet.DispatcherServlet : Initializing Servlet 'dispatcherServlet'
2026-08-20T08:55:49:299-04:00 INFO 32894 --- [nio-8080-exec-1] o.a.w.servlet.DispatcherServlet : Complete initialization in 0 ms
2026-08-20T08:55:49:537-04:00 INFO 32894 --- [nio-8080-exec-1] o.s.d.c.c.s.TaskExecutorJobLauncher : Job: [SimpleJob: [name=cuentaAnualJob]] launched with the following parameters: [{"time":{"value=1787230547201, type=class java.lang.Long, identifying=true}}]
2026-08-20T08:55:49:592-04:00 INFO 32894 --- [nio-8080-exec-1] o.d.migration.Listener.BancoJobListener :
=====
INICIO DE JOB: [cuentaAnualJob] (ID: 3)
Parámetros: {"time":{"value=1787230547201, type=class java.lang.Long, identifying=true}}
=====
2026-08-20T08:55:49:623-04:00 INFO 32894 --- [nio-8080-exec-1] o.s.batch.core.job.SimpleStepHandler : Executing step: [cuentaAnualStep]
2026-08-20T08:55:49:649-04:00 WARN 32894 --- [nio-8080-exec-1] o.s.batch.core.step.item.ChunkMonitor : No ItemReader set (must be concurrent step), so ignoring offset data.
2026-08-20T08:55:49:743-04:00 WARN 32894 --- [batch-thread-1] o.s.batch.core.step.item.ChunkMonitor : ItemStream was opened in a different thread. Restart data could be compromised.
2026-08-20T08:55:49:743-04:00 WARN 32894 --- [batch-thread-1] o.s.batch.core.step.item.ChunkMonitor : ItemStream was opened in a different thread. Restart data could be compromised.
2026-08-20T08:55:49:743-04:00 WARN 32894 --- [batch-thread-1] o.s.batch.core.step.item.ChunkMonitor : ItemStream was opened in a different thread. Restart data could be compromised.
Hibernate: insert into cuentas_anuales (cuenta_id,descripcion,fecha,monto,transaccion) values (?,?,?,?)
Hibernate: insert into cuentas_anuales (cuenta_id,descripcion,fecha,monto,transaccion) values (?,?,?,?)
Hibernate: insert into cuentas_anuales (cuenta_id,descripcion,fecha,monto,transaccion) values (?,?,?,?)
Hibernate: insert into cuentas_anuales (cuenta_id,descripcion,fecha,monto,transaccion) values (?,?,?,?)
Hibernate: insert into cuentas_anuales (cuenta_id,descripcion,fecha,monto,transaccion) values (?,?,?,?)
Hibernate: insert into cuentas_anuales (cuenta_id,descripcion,fecha,monto,transaccion) values (?,?,?,?)
Hibernate: insert into cuentas_anuales (cuenta_id,descripcion,fecha,monto,transaccion) values (?,?,?,?)
2026-08-20T08:55:49:977-04:00 INFO 32894 --- [nio-8080-exec-1] o.s.batch.core.step.AbstractStep : Step: [cuentaAnualStep] executed in 370ms
2026-08-20T08:55:50:026-04:00 INFO 32894 --- [nio-8080-exec-1] o.d.migration.Listener.BancoJobListener :
=====
RESULTADO JOB: [cuentaAnualJob] -> estado: COMPLETED
=====
2026-08-20T08:55:50:034-04:00 INFO 32894 --- [nio-8080-exec-1] o.s.d.c.c.s.TaskExecutorJobLauncher : Job: [SimpleJob: [name=cuentaAnualJob]] completed with the following parameters: [{"time":{"value=1787230547201, type=class java.lang.Long, identifying=true}}] and the following status: [COMPLETED] in 471ms
2026-08-20T08:55:50:438-04:00 INFO 32894 --- [nio-8080-exec-2] o.s.d.c.c.s.TaskExecutorJobLauncher : Job: [SimpleJob: [name=interesesJob]] launched with the following parameters: [{"time":{"value=1787230547396, type=class java.lang.Long, identifying=true}}]
2026-08-20T08:55:50:476-04:00 INFO 32894 --- [nio-8080-exec-2] o.d.migration.Listener.BancoJobListener :
=====
INICIO DE JOB: [intereses] (ID: 2)
Parámetros: {"time":{"value=1787230547396, type=class java.lang.Long, identifying=true}}
=====
2026-08-20T08:55:50:485-04:00 INFO 32894 --- [nio-8080-exec-2] o.s.batch.core.job.SimpleStepHandler : Executing step: [interesesStep]
Hibernate: insert into intereses (id_cuenta,monto_base,monto_interes,periodo,tasa_interes) values (?,?,?,?)
Hibernate: insert into intereses (id_cuenta,monto_base,monto_interes,periodo,tasa_interes) values (?,?,?,?)
Hibernate: insert into intereses (id_cuenta,monto_base,monto_interes,periodo,tasa_interes) values (?,?,?,?)
Hibernate: insert into intereses (id_cuenta,monto_base,monto_interes,periodo,tasa_interes) values (?,?,?,?)
Hibernate: insert into intereses (id_cuenta,monto_base,monto_interes,periodo,tasa_interes) values (?,?,?,?)
2026-08-20T08:55:50:615-04:00 INFO 32894 --- [nio-8080-exec-2] o.s.batch.core.step.AbstractStep : Step: [interesesStep] executed in 97ms
2026-08-20T08:55:50:635-04:00 INFO 32894 --- [nio-8080-exec-2] o.d.migration.Listener.BancoJobListener :
=====
RESULTADO JOB: [intereses] -> estado: COMPLETED
=====
2026-08-20T08:55:50:651-04:00 INFO 32894 --- [nio-8080-exec-2] o.s.d.c.c.s.TaskExecutorJobLauncher : Job: [SimpleJob: [name=interesesJob]] completed with the following parameters: [{"time":{"value=1787230547396, type=class java.lang.Long, identifying=true}}] and the following status: [COMPLETED] in 120ms
2026-08-20T08:55:50:776-04:00 INFO 32894 --- [nio-8080-exec-3] o.s.d.c.c.s.TaskExecutorJobLauncher : Job: [SimpleJob: [name=transaccionesJob]] launched with the following parameters: [{"time":{"value=1787230550217, type=class java.lang.Long, identifying=true}}]
2026-08-20T08:55:50:792-04:00 INFO 32894 --- [nio-8080-exec-3] o.d.migration.Listener.BancoJobListener :
=====
INICIO DE JOB: [transacciones] (ID: 3)
Parámetros: {"time":{"value=1787230550217, type=class java.lang.Long, identifying=true}}
=====
2026-08-20T08:55:50:833-04:00 INFO 32894 --- [nio-8080-exec-3] o.s.batch.core.job.SimpleStepHandler : Executing step: [transaccionesStep]
2026-08-20T08:55:50:835-04:00 WARN 32894 --- [nio-8080-exec-3] o.s.batch.core.step.item.ChunkMonitor : No ItemReader set (must be concurrent step), so ignoring offset data.
2026-08-20T08:55:50:889-04:00 WARN 32894 --- [batch-thread-1] o.s.batch.core.step.item.ChunkMonitor : ItemStream was opened in a different thread. Restart data could be compromised.
2026-08-20T08:55:50:889-04:00 WARN 32894 --- [batch-thread-1] o.s.batch.core.step.item.ChunkMonitor : ItemStream was opened in a different thread. Restart data could be compromised.
2026-08-20T08:55:50:889-04:00 WARN 32894 --- [batch-thread-1] o.s.batch.core.step.item.ChunkMonitor : ItemStream was opened in a different thread. Restart data could be compensated.
Hibernate: select ti.0.id,ti.0.fecha,ti.0.monto,ti.0.tipo from transacciones ti.0 where ti.0.id=?
Hibernate: select ti.0.id,ti.0.fecha,ti.0.monto,ti.0.tipo from transacciones ti.0 where ti.0.id=?
Hibernate: select ti.0.id,ti.0.fecha,ti.0.monto,ti.0.tipo from transacciones ti.0 where ti.0.id=?
Hibernate: select ti.0.id,ti.0.fecha,ti.0.monto,ti.0.tipo from transacciones ti.0 where ti.0.id=?
Hibernate: select ti.0.id,ti.0.fecha,ti.0.monto,ti.0.tipo from transacciones ti.0 where ti.0.id=?
Hibernate: select ti.0.id,ti.0.fecha,ti.0.monto,ti.0.tipo from transacciones ti.0 where ti.0.id=?
Hibernate: select ti.0.id,ti.0.fecha,ti.0.monto,ti.0.tipo from transacciones ti.0 where ti.0.id=?
Hibernate: insert into transacciones (fecha,monto,tipo,id) values (?,?,?,?)
Hibernate: insert into transacciones (fecha,monto,tipo,id) values (?,?,?,?)
Hibernate: insert into transacciones (fecha,monto,tipo,id) values (?,?,?,?)
Hibernate: insert into transacciones (fecha,monto,tipo,id) values (?,?,?,?)
Hibernate: insert into transacciones (fecha,monto,tipo,id) values (?,?,?,?)
Hibernate: insert into transacciones (fecha,monto,tipo,id) values (?,?,?,?)
2026-08-20T08:55:50:879-04:00 INFO 32894 --- [nio-8080-exec-3] o.s.batch.core.step.AbstractStep : Step: [transaccionesStep] executed in 240ms
2026-08-20T08:55:50:905-04:00 INFO 32894 --- [nio-8080-exec-3] o.d.migration.Listener.BancoJobListener :
=====
RESULTADO JOB: [transacciones] -> estado: COMPLETED
=====
2026-08-20T08:55:50:919-04:00 INFO 32894 --- [nio-8080-exec-3] o.s.d.c.c.s.TaskExecutorJobLauncher : Job: [SimpleJob: [name=transaccionJob]] completed with the following parameters: [{"time":{"value=1787230550217, type=class java.lang.Long, identifying=true}}] and the following status: [COMPLETED] in 327ms
```

Parte 2 (Intereses y Transacciones):

```

Hibernate: insert into intereses (id_cuenta,monto_base,monto_interes,periodo,tasa_interes) values (?,?,?,?)
Hibernate: insert into intereses (id_cuenta,monto_base,monto_interes,periodo,tasa_interes) values (?,?,?,?)
Hibernate: insert into intereses (id_cuenta,monto_base,monto_interes,periodo,tasa_interes) values (?,?,?,?)
Hibernate: insert into intereses (id_cuenta,monto_base,monto_interes,periodo,tasa_interes) values (?,?,?,?)
Hibernate: insert into intereses (id_cuenta,monto_base,monto_interes,periodo,tasa_interes) values (?,?,?,?)
Hibernate: insert into intereses (id_cuenta,monto_base,monto_interes,periodo,tasa_interes) values (?,?,?,?)
2026-08-20T08:55:50:615-04:00 INFO 32894 --- [nio-8080-exec-2] o.s.batch.core.step.AbstractStep : Step: [interesesStep] executed in 97ms
2026-08-20T08:55:50:635-04:00 INFO 32894 --- [nio-8080-exec-2] o.d.migration.Listener.BancoJobListener :
=====
RESULTADO JOB: [intereses] -> estado: COMPLETED
=====
2026-08-20T08:55:50:651-04:00 INFO 32894 --- [nio-8080-exec-2] o.s.d.c.c.s.TaskExecutorJobLauncher : Job: [SimpleJob: [name=interesesJob]] completed with the following parameters: [{"time":{"value=1787230547396, type=class java.lang.Long, identifying=true}}] and the following status: [COMPLETED] in 120ms
2026-08-20T08:55:50:776-04:00 INFO 32894 --- [nio-8080-exec-3] o.s.d.c.c.s.TaskExecutorJobLauncher : Job: [SimpleJob: [name=transaccionesJob]] launched with the following parameters: [{"time":{"value=1787230550217, type=class java.lang.Long, identifying=true}}]
2026-08-20T08:55:50:792-04:00 INFO 32894 --- [nio-8080-exec-3] o.d.migration.Listener.BancoJobListener :
=====
INICIO DE JOB: [transacciones] (ID: 3)
Parámetros: {"time":{"value=1787230550217, type=class java.lang.Long, identifying=true}}
=====
2026-08-20T08:55:50:833-04:00 INFO 32894 --- [nio-8080-exec-3] o.s.batch.core.job.SimpleStepHandler : Executing step: [transaccionesStep]
2026-08-20T08:55:50:835-04:00 WARN 32894 --- [nio-8080-exec-3] o.s.batch.core.step.item.ChunkMonitor : No ItemReader set (must be concurrent step), so ignoring offset data.
2026-08-20T08:55:50:889-04:00 WARN 32894 --- [batch-thread-1] o.s.batch.core.step.item.ChunkMonitor : ItemStream was opened in a different thread. Restart data could be compromised.
2026-08-20T08:55:50:889-04:00 WARN 32894 --- [batch-thread-1] o.s.batch.core.step.item.ChunkMonitor : ItemStream was opened in a different thread. Restart data could be compromised.
2026-08-20T08:55:50:889-04:00 WARN 32894 --- [batch-thread-1] o.s.batch.core.step.item.ChunkMonitor : ItemStream was opened in a different thread. Restart data could be compensated.
Hibernate: select ti.0.id,ti.0.fecha,ti.0.monto,ti.0.tipo from transacciones ti.0 where ti.0.id=?
Hibernate: select ti.0.id,ti.0.fecha,ti.0.monto,ti.0.tipo from transacciones ti.0 where ti.0.id=?
Hibernate: select ti.0.id,ti.0.fecha,ti.0.monto,ti.0.tipo from transacciones ti.0 where ti.0.id=?
Hibernate: select ti.0.id,ti.0.fecha,ti.0.monto,ti.0.tipo from transacciones ti.0 where ti.0.id=?
Hibernate: select ti.0.id,ti.0.fecha,ti.0.monto,ti.0.tipo from transacciones ti.0 where ti.0.id=?
Hibernate: select ti.0.id,ti.0.fecha,ti.0.monto,ti.0.tipo from transacciones ti.0 where ti.0.id=?
Hibernate: select ti.0.id,ti.0.fecha,ti.0.monto,ti.0.tipo from transacciones ti.0 where ti.0.id=?
Hibernate: insert into transacciones (fecha,monto,tipo,id) values (?,?,?,?)
Hibernate: insert into transacciones (fecha,monto,tipo,id) values (?,?,?,?)
Hibernate: insert into transacciones (fecha,monto,tipo,id) values (?,?,?,?)
Hibernate: insert into transacciones (fecha,monto,tipo,id) values (?,?,?,?)
Hibernate: insert into transacciones (fecha,monto,tipo,id) values (?,?,?,?)
2026-08-20T08:55:50:879-04:00 INFO 32894 --- [nio-8080-exec-3] o.s.batch.core.step.AbstractStep : Step: [transaccionesStep] executed in 240ms
2026-08-20T08:55:50:905-04:00 INFO 32894 --- [nio-8080-exec-3] o.d.migration.Listener.BancoJobListener :
=====
RESULTADO JOB: [transacciones] -> estado: COMPLETED
=====
2026-08-20T08:55:50:919-04:00 INFO 32894 --- [nio-8080-exec-3] o.s.d.c.c.s.TaskExecutorJobLauncher : Job: [SimpleJob: [name=transaccionJob]] completed with the following parameters: [{"time":{"value=1787230550217, type=class java.lang.Long, identifying=true}}] and the following status: [COMPLETED] in 327ms
```

6.2 Persistencia de Datos en Base de Datos (MySQL Workbench)

Tabla transacciones

Demuestra que los datos del archivo CSV fueron leídos, procesados por *chunks* y almacenados correctamente en la base de datos `bancoxyz\_db`, visualizándose tanto los registros mapeados como su estructura relacional.

The screenshot shows the MySQL Workbench interface with the 'bancoxyz_db' schema selected in the left-hand pane. The central pane displays a SQL query window with two queries:

```
1 • SELECT COUNT(*) AS total_transacciones FROM transacciones;
2 • SELECT * FROM transacciones LIMIT 5;
```

The 'Result Grid' shows the results of the second query, displaying 5 rows of transaction data:

id	fecha	monto	tipo
1	2024-01-01	1000	debito
2	2024-01-02	1500	credito
4	2024-01-03	0	debito
5	2024-01-04	800	credito
6	2024-01-05	700	debito

The bottom pane shows the 'Output' tab with a table of actions and their results:

#	Time	Action	Message	Duration / Fetch
11	08:58:56	SELECT * FROM transacciones LIMIT 5	5 row(s) returned	0.000 sec / 0.000 sec
12	08:59:14	SELECT COUNT(*) AS total_intereses FROM intereses LIMIT 0, 1000	1 row(s) returned	0.019 sec / 0.000 sec
13	08:59:14	SELECT * FROM intereses LIMIT 5	5 row(s) returned	0.000 sec / 0.000 sec
14	08:59:37	SELECT COUNT(*) AS total_cuentas FROM cuentas_ anuales LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec
15	08:59:37	SELECT * FROM cuentas_ anuales LIMIT 5	5 row(s) returned	0.000 sec / 0.000 sec

Tabla intereses

Muestra el resultado de las consultas ejecutadas en MySQL Workbench (bancoxyz_db), comprobando que los registros provenientes del archivo CSV de intereses fueron leídos, procesados y almacenados correctamente con sus respectivas columnas (id_cuenta, monto_base, monto_interes, periodo y tasa_interes).

The screenshot shows the MySQL Workbench interface. On the left, the 'SCHEMAS' pane lists various databases, with 'bancoxyz_db' selected. The main editor displays two SQL queries in 'SQL File 4':

```
1 • SELECT COUNT(*) AS total_intereses FROM intereses;
2 • SELECT * FROM intereses LIMIT 5;
```

The 'Result Grid' shows the results of the second query, displaying 5 rows of data from the 'intereses' table. The columns are: id, id_cuenta, monto_base, monto_interes, periodo, and tasa_interes.

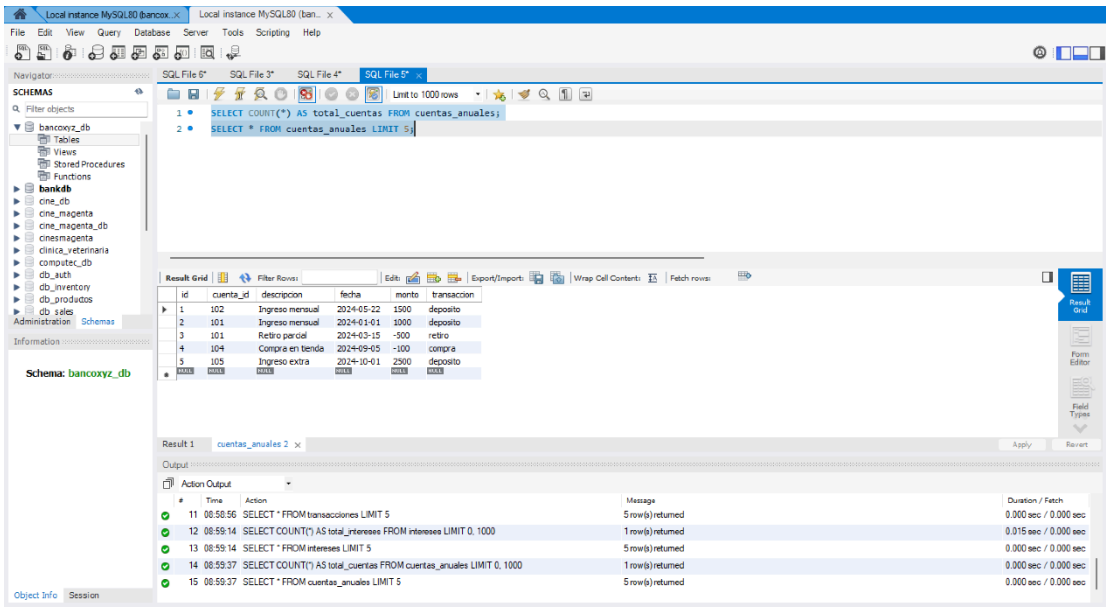
	id	id_cuenta	monto_base	monto_interes	periodo	tasa_interes
1	101	8000	125	2026-06	2.5	
2	102	8000	360	2026-06	4.5	
3	103	12000	540	2026-06	4.5	
4	105	7000	175	2026-06	2.5	
5	106	9000	125	2026-06	2.5	

Below the grid, the 'Action Output' pane shows the execution log for the queries. The log indicates that the first query returned 1 row and the second query returned 5 rows.

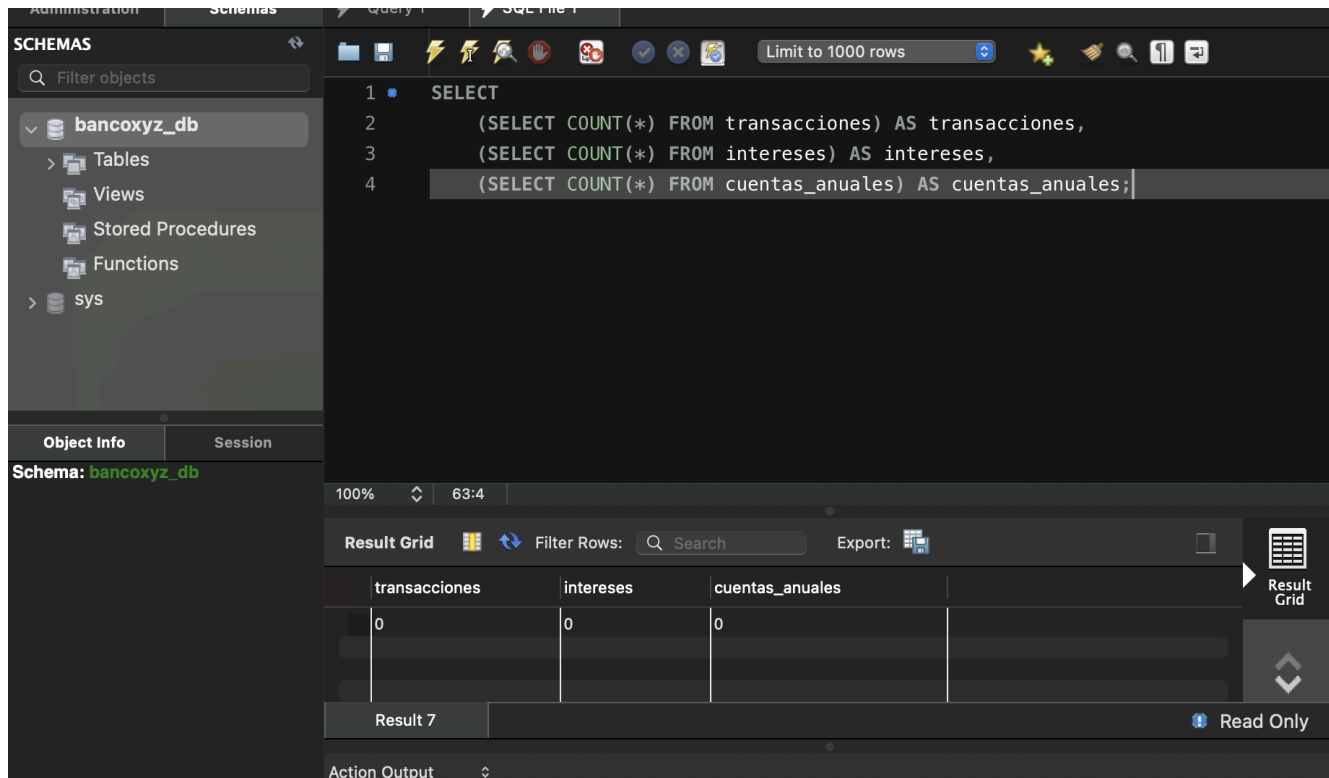
#	Time	Action	Message	Duration / Fetch
11	08:58:56	SELECT * FROM transacciones LIMIT 5	5 row(s) returned	0.000 sec / 0.000 sec
12	08:59:14	SELECT COUNT(*) AS total_intereses FROM intereses LIMIT 0, 1000	1 row(s) returned	0.015 sec / 0.000 sec
13	08:59:14	SELECT * FROM intereses LIMIT 5	5 row(s) returned	0.000 sec / 0.000 sec
14	08:59:37	SELECT COUNT(*) AS total_cuentas FROM cuentas_anuales LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec
15	08:59:37	SELECT * FROM cuentas_anuales LIMIT 5	5 row(s) returned	0.000 sec / 0.000 sec

Tabla cuentas_anuales.

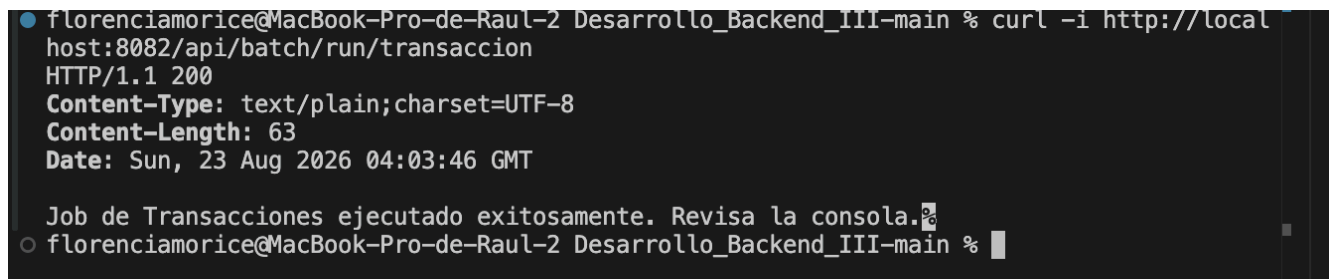
Muestra el resultado de las consultas de verificación ejecutadas en MySQL Workbench dentro del esquema bancoxyz_db, comprobando que los estados de cuenta anuales provenientes de la migración batch fueron almacenados con éxito y muestran de forma correcta sus campos (id, cuenta_id, descripcion, fecha, monto, transaccion).



Estado_inicial_mysql.png, Base de datos MySQL limpia antes de la ejecución de los Jobs:
0 registros en las tablas de negocio:



Ejecución del Job de transacciones: se evidencia la ejecución exitosa del transaccionJob, con respuesta HTTP 200 y estado COMPLETED, confirmando el procesamiento correcto de las transacciones en la base de datos MySQL.



Ejecución exitosa del Job de Intereses. La respuesta HTTP 200 confirma que el proceso fue ejecutado correctamente:

```
• florenciamorice@MacBook-Pro-de-Raul-2 Desarrollo_Backend_III-main % curl -i http://localhost:8082/api/batch/run/interes
HTTP/1.1 200
Content-Type: text/plain;charset=UTF-8
Content-Length: 59
Date: Sun, 23 Aug 2026 04:11:42 GMT

Job de Intereses ejecutado exitosamente. Revisa la consola.%
○ florenciamorice@MacBook-Pro-de-Raul-2 Desarrollo_Backend_III-main %
```

```
2026-08-23T00:11:42.320-04:00 INFO 14100 --- [nio-8082-exec-2] o.s.batch.core.step.AbstractStep : Step: [interesStep] executed in 48ms
[BancoJobListener] Job finalizado: interesJob
[BancoJobListener] Inicio: 2026-08-23T04:11:42.265574Z
[BancoJobListener] Fin: 2026-08-23T04:11:42.327656Z
[BancoJobListener] Duracion: 62 ms
[BancoJobListener] Estado: COMPLETED
[BancoJobListener] Registros procesados: 8
[BancoJobListener] Registros omitidos: 0
2026-08-23T00:11:42.330-04:00 INFO 14100 --- [nio-8082-exec-2] o.s.b.c.l.s.TaskExecutor
```

Ejecución exitosa del Job de Consolidación Anual. La respuesta HTTP 200 confirma que el proceso fue ejecutado correctamente:

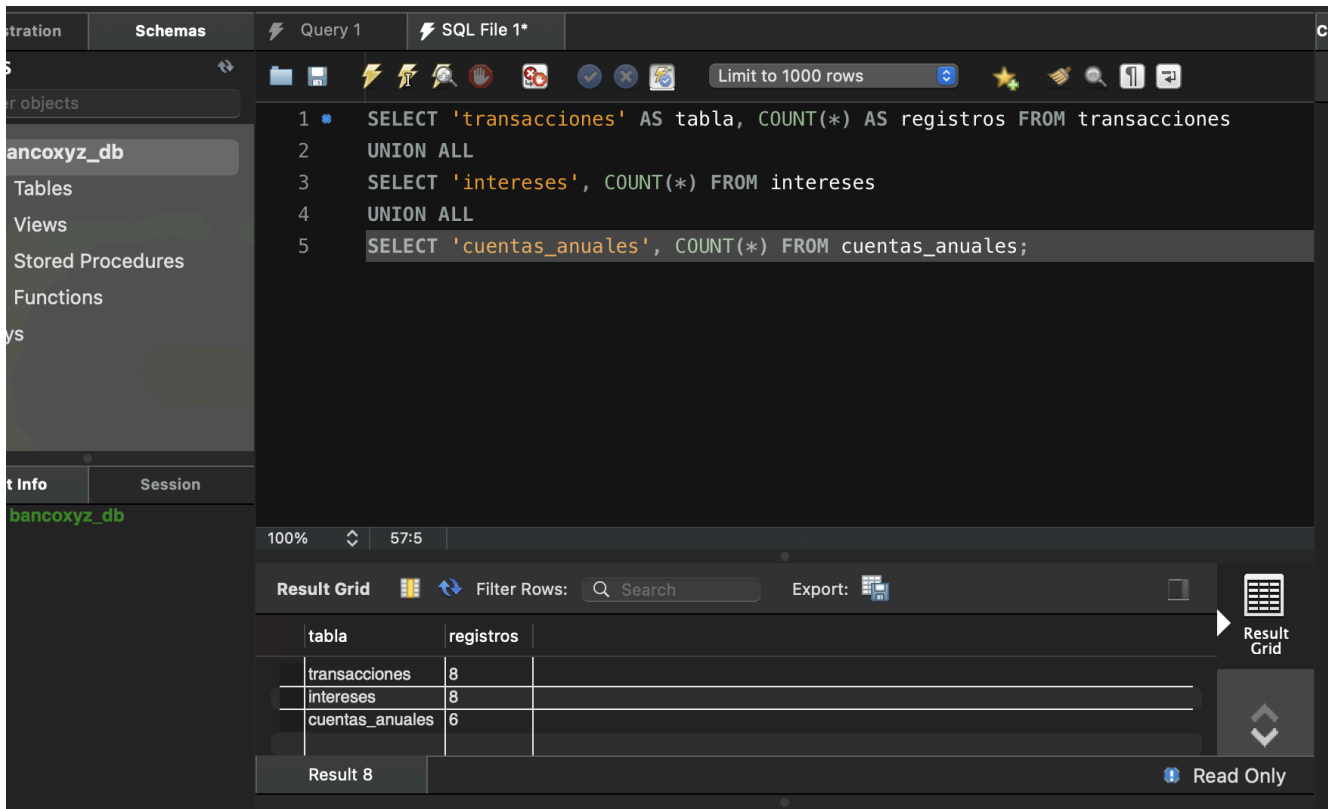
```
● florenciamorice@MacBook-Pro-de-Raul-2 Desarrollo_Backend_III-main % curl -i http://localhost:8082/api/batch/run/cuenta-anual
HTTP/1.1 200
Content-Type: text/plain;charset=UTF-8
Content-Length: 62
Date: Sun, 23 Aug 2026 04:14:38 GMT
```

Job de Cuenta Anual ejecutado exitosamente. Revisa la consola.

```
○ florenciamorice@MacBook-Pro-de-Raul-2 Desarrollo_Backend_III-main %
```

```
2026-08-23T00:14:38.249-04:00 INFO 14100 --- [nio-8082-exec-3] o.s.batch.core.step.AbstractStep : Step: [cuentaAnualStep] executed in 113ms
[BancoJobListener] Job finalizado: cuentaAnualJob
[BancoJobListener] Inicio: 2026-08-23T04:14:38.130219Z
[BancoJobListener] Fin: 2026-08-23T04:14:38.253360Z
[BancoJobListener] Duracion: 123 ms
[BancoJobListener] Estado: COMPLETED
[BancoJobListener] Registros procesados: 6
[BancoJobListener] Registros omitidos: 3
2026-08-23T00:14:38.255-04:00 INFO 14100 --- [nio-8082-exec-3] o.s.b.c.l.s.TaskExecutorJobLauncher : Job: [SimpleJob: [name=cuentaAnualJob]] completed with the following parameters: [{ 'time': '{value=1787458478052, type=class java.lang.Long, identifying=true}' }] and the following status: [COMPLETED] in 126ms
```

Verificación final de persistencia: se consulta la cantidad de registros almacenados en las tres tablas de negocio después de la ejecución de los Jobs, comprobando que los datos fueron procesados y persistidos correctamente en MySQL:



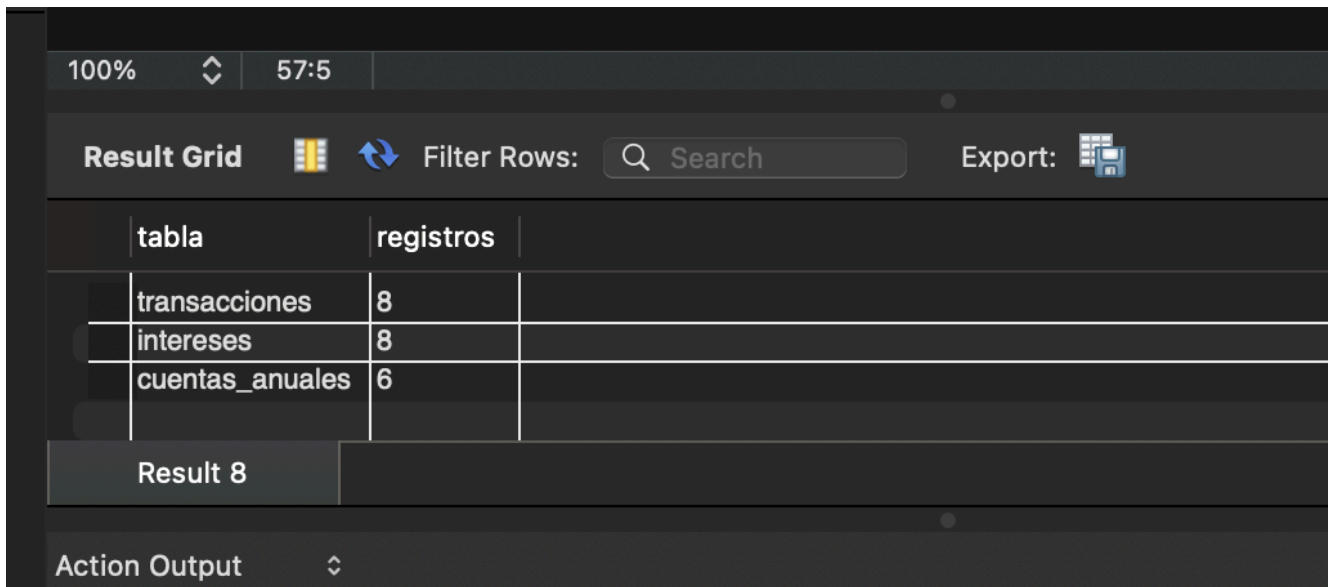
The screenshot shows a SQL IDE interface. On the left, a sidebar displays the database structure for 'bancoxyz_db', including Tables, Views, Stored Procedures, and Functions. The main area shows a SQL query in 'Query 1' that counts records in three tables: 'transacciones', 'intereses', and 'cuentas_anuales'. The query is as follows:

```
1 SELECT 'transacciones' AS tabla, COUNT(*) AS registros FROM transacciones
2 UNION ALL
3 SELECT 'intereses', COUNT(*) FROM intereses
4 UNION ALL
5 SELECT 'cuentas_anuales', COUNT(*) FROM cuentas_anuales;
```

Below the query editor, the 'Result Grid' is displayed, showing the results of the query. The grid has two columns: 'tabla' and 'registros'. The results are as follows:

tabla	registros
transacciones	8
intereses	8
cuentas_anuales	6

The interface also shows a 'Session' tab, a 'Filter Rows' search bar, and an 'Export' button. The status bar at the bottom indicates 'Result 8' and 'Read Only'.



This is a close-up view of the 'Result Grid' from the previous screenshot. It shows the same data: three rows representing the tables 'transacciones', 'intereses', and 'cuentas_anuales' with their respective record counts of 8, 8, and 6. The interface includes a 'Filter Rows' search bar and an 'Export' button. The status bar at the bottom indicates 'Result 8' and 'Read Only'.



Reservados todos los derechos Fundación Instituto Profesional Duoc UC. No se permite copiar, reproducir, reeditar, descargar, publicar, emitir, difundir, de forma total o parcial la presente obra, ni su incorporación a un sistema informático, ni su transmisión en cualquier forma o por cualquier medio (electrónico, mecánico, fotocopia, grabación u otros) sin autorización previa y por escrito de Fundación Instituto Profesional Duoc UC. La infracción de dichos derechos puede constituir un delito contra la propiedad intelectual.